

```
In [ ]: #RunningSum/Prefix Sum
class solution:
    def RunningSum(self,arr:list[int])>->list[int]:
        ans=[]
        ans.append(arr[0])

        for i in range(1,len(arr)):
            x=ans[i-1]+arr[i]
            ans.append(x)
        return ans
obj=solution()
result=obj.RunningSum([3,1,8,2,10,5])
print(f'Result is:{result}')
```

Result is:[3, 4, 12, 14, 24, 29]

```
In [ ]: #Remove duplicates in an array
class solution:
    def RemoveDuplicates(self,arr:list[int])>->list[int]:
        ans=[]
        ans.append(arr[0])
        n=len(arr)

        for i in range(1,n):
            if arr[i]!=arr[i-1]:
                ans.append(arr[i])
        return ans

obj=solution()
result=obj.RemoveDuplicates([1,2,2,3,3,4,4,5,6,9,7,7,8])
print(f'Result is:{result}')
```

Result is:[1, 2, 3, 4, 5, 6, 9, 7, 8]

```
In [14]: #Remove duplicates in an array [INPLACE] without creating new DS or array
class solution:
    def RemoveDuplicateandC0unt(self,arr:list[int])>->int:
        start=0

        n=len(arr)
        for i in range(1,n):
            if arr[start]!=arr[i]:
                start+=1
                arr[start]=arr[i]
        return start
obj=solution()
obj.RemoveDuplicateandC0unt([1,2,2,3,3,4,4,5,6,7,7,8])

#Remove Duplicates II , Atmost 2 same element is okay
class solution:
    def RemoveDuplicates2(self,arr:list[int])>->int:
        start=1

        n=len(arr)

        for i in range(2,n):
            if arr[i]!=arr[start-1]:
                start+=1
                arr[start]=arr[i]
```

```

        return start+1
obj=solution()
obj.RemoveDuplicates2([1,2,2,3,3,4,4,5,6,7,7,8])

```

Out[14]: 12

```

In [4]: #Sort array by parity
class solution:
    def SortByParity(self,arr:list[int])>list[int]:
        start=0
        for i in range(len(arr)):
            if arr[i]%2==0:
                temp=arr[i]
                arr[i]=arr[start]
                arr[start]=temp
                start+=1
        return arr
obj=solution()
result=obj.SortByParity([5,1,6,2,7,4,3])
print(f'Result is:{result}')

```

Result is:[6, 2, 4, 1, 7, 5, 3]

```

In [8]: #Maximum sum Subarray--> Kadaneis algorithm
class solution:
    def MaximumSubarray(self,arr:list[int])>int:
        curr_sum=0
        max_sum=arr[0]

        for i in range(len(arr)):
            curr_sum+=arr[i]
            if curr_sum>max_sum:
                max_sum=curr_sum
            if curr_sum<0:
                curr_sum=0
        return max_sum
obj=solution()
result=obj.MaximumSubarray([5,-1,-7,6,2,-3,5,-10,2])
print(f'Result is:{result}')

```

Result is:10

```

In [14]: #Best time to but and sell product
class solution:
    def BestTime(self,arr:list[int])>int:
        profit=0
        min_price=arr[0]

        for i in range(1,len(arr)):
            curr_profit=arr[i]-min_price
            if curr_profit>profit:
                profit=curr_profit
            min_price=min(min_price,arr[i])
        return profit
obj=solution()
result=obj.BestTime([7,1,5,3,6,4])
print(f'profit is:{result}')

```

profit is:5

```
In [20]: #Max wealth of person
class solution:
    def MaxWealth(self,accounts:list[list[int]]->int:
        ans=0

        for account in accounts:
            ans=max(ans,sum(account))
        return ans
obj=solution()
result=obj.MaxWealth([[1,2,3],[4,5,6]])
print(f'Result is:{result}')
```

Result is:15

```
In [ ]: #Spiral Matrix
class solution:
    def SpiralMatrix(self,matrix:list[list[int]]->list[int]:

        ans=[]

        n=len(matrix)
        m=len(matrix[0])
        total=n*m

        rowstart=0
        rowend=n-1
        colstart=0
        colend=m-1

        count=0

        while total>count:
            #rowstart-> colstart-->colend
            for i in range(colstart,colend+1):
                ans.append(matrix[rowstart][i])
                count+=1
            rowstart+=1

            if count==total:
                break

            #colend-> rowstart-->rowend
            for i in range(rowstart,rowend+1):
                ans.append(matrix[i][colend])
                count+=1
            colend-=1

            if count==total:
                break

            #rowend-> colend-->colstart
            for i in range(colend,colstart-1,-1):
                ans.append(matrix[rowend][i])
                count+=1
            rowend-=1

            if count==total:
                break
```

```

        #colstart--> rowend->rowstart
        for i in range(rowend,rowstart-1,-1):
            ans.append(matrix[i][colstart])
            count+=1
            colstart+=1

        if count==total:
            break

    return ans
obj=solution()
result=obj.SpiralMatrix([[1,2,3,4,5],[6,7,8,9,10],[11,12,13,14,15]])
print(f'Result is:{result}')

```

Result is:[1, 2, 3, 4, 5, 10, 15, 14, 13, 12, 11, 6, 7, 8, 9]

In [22]: *#Defanging an IP address*

```

class solution:
    def DefangAddress(self,address:str)->str:
        ans=''
        for i in address:
            if i!=".":
                ans+=i
            else:
                ans+="[.]"
        return ans
obj=solution()
result=obj.DefangAddress("121.1.34.121")
print(f'Result is:{result}')

```

Result is:121[.]1[.]34[.]121

In [31]: *#Valid Palindrome string.*

```

class solution:
    def IsAlphaNumeric(self,s):
        x=ord(s)
        if 97<=x<=122 or 65<=x<=90 or 48<=x<=57:
            return True
        return False
    def IsPalindrome(self,s:str)->bool:
        s=s.lower()
        i=0
        j=len(s)-1
        while i<j:
            if not self.IsAlphaNumeric(s[i]):
                i+=1
            elif not self.IsAlphaNumeric(s[j]):
                j-=1
            elif s[i]==s[j]:
                i+=1
                j-=1
            else:
                return False
        return True
obj=solution()
result=obj.IsPalindrome("A man, a plan, a canal: panama")
print(f'Result is:{result}')

```

Result is:True

```
In [35]: #Reverse String
class solution:
    def ReverseString(self,s:list[str])>->list[str]:
        i=0
        j=len(s)-1

        while i<j:
            temp=s[i]
            s[i]=s[j]
            s[j]=temp
            i+=1
            j-=1
        return s

obj=solution()
result=obj.ReverseString(['A',"D","H","I",'L'])
print(f'Reverse string is:{result}')
```

Reverse string is:['L', 'I', 'H', 'D', 'A']

```
In [ ]: #Length of last word
class solution:
    def LengthOfLastWord(self,s:str)>->int:
        s=s.strip()
        i=-1
        n=len(s)

        while i>(-1*n) and s[i]!=" ":
            i-=1
        i+=1
        return -1*i

obj=solution()
result=obj.LengthOfLastWord(" HE is Flying ")
print(f'Result is:{result}')
```

Result is:6

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click

View Jupyter